

Vertex Threading / Concurrency Model Specification

Version: 1.0

Status: Draft

1. Objetivo

O Threading / Concurrency Model do Vertex define como tarefas são executadas simultaneamente, garantindo eficiência, isolamento e previsibilidade em um sistema baseado em plugins.

Seu objetivo é fornecer um modelo unificado de concorrência que funcione de forma consistente entre:

- Execution Runtime;
- Scheduler;
- UI Runtime;
- Core Services;
- Plugins.

O modelo é baseado principalmente em Kotlin Coroutines, com abstrações adicionais do Core para controle, isolamento e orçamento de execução.

2. Responsabilidades

O modelo de concorrência é responsável por:

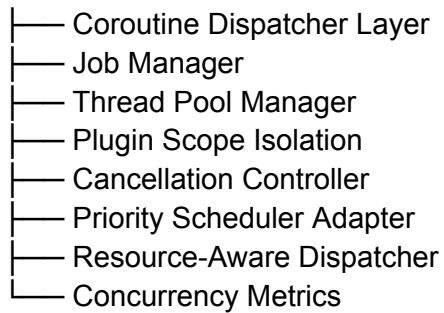
- definir como tarefas são executadas simultaneamente;
- gerenciar criação e destruição de jobs;
- controlar cancelamento cooperativo;
- limitar concorrência por plugin;
- coordenar execução entre subsistemas;
- evitar bloqueios do sistema;
- garantir fairness entre tarefas;
- integrar com Scheduler e Execution Runtime;
- respeitar limites de recursos.

3. Arquitetura

Vertex Core

↓

Concurrency Layer



Kotlin Coroutines



Native Threads (Android Runtime)

4. Componentes

Coroutine Dispatcher Layer

Define os contextos de execução:

- Main (UI)
- IO
- Default (CPU-bound)
- Plugin-Specific Dispatcher

Job Manager

Controla o ciclo de vida de jobs.

Thread Pool Manager

Gerencia pools de threads otimizados para:

- UI tasks;
- background tasks;
- heavy computation;

- system tasks.

Plugin Scope Isolation

Garante que cada plugin tenha seu próprio escopo de concorrência.

Cancellation Controller

Implementa cancelamento cooperativo.

Priority Scheduler Adapter

Integra o Scheduler com o modelo de concorrência.

Resource-Aware Dispatcher

Adapta execução com base em:

- CPU;
- memória;
- bateria;
- carga do sistema.

Concurrency Metrics

Monitora comportamento do sistema concorrente.

5. Fluxos

Execução de tarefa

Plugin / Core

↓

Coroutine Scope

↓

Dispatcher Selection

↓

Job Manager

↓

Thread Pool

↓

Execution Runtime

Cancelamento

Scheduler / Power Manager

↓

Cancellation Controller

↓

Job Manager

↓

Coroutine Cancel

Escalonamento

Scheduler

↓

Priority Adapter

↓

Dispatcher Layer

↓

Execution

6. APIs

Escopos

- createScope()
- pluginScope()
- systemScope()

Execução

- launch()
- async()
- withContext()

Controle

- cancel()
- cancelAll()
- isActive()

Monitoramento

- jobStatus()
- activeJobs()

7. Estados

Job State

Created

↓

Active



Suspending



Suspended



Completed



Cancelled



Failed

Dispatcher State

- Idle
- Active
- Overloaded
- Throttled

8. Políticas

Cooperação

Todas as tarefas devem ser cooperativas.

Cancelamento obrigatório

Tarefas devem responder a cancelamento de forma segura.

Isolamento por plugin

Nenhum plugin pode interferir no fluxo concorrente de outro.

Limitação de concorrência

Cada plugin possui limites de:

- threads simultâneas;
- jobs ativos;
- uso de CPU.

Prioridade adaptativa

Tarefas críticas podem preemptar tarefas de baixa prioridade.

Eficiência energética

O modelo deve reduzir atividade em segundo plano quando possível.

9. Integrações

O Threading Model integra-se com:

- Execution Runtime;
- Scheduler;
- Plugin Manager;
- Plugin API;
- Plugin Context;
- Resource Manager;
- Power Management;
- State Engine;
- Event System;
- Capability System;
- UI Runtime;
- Logging Framework;
- Security Framework.

10. Métricas

O sistema coleta:

- número de jobs ativos;
- tempo médio de execução;
- tempo de espera;
- uso de threads;
- cancelamentos;
- deadlocks evitados;
- throttling events;
- consumo de CPU por plugin;
- tempo em idle;
- overload frequency.

11. Exemplos

Execução paralela em plugin

1. Plugin inicia múltiplas tarefas.
2. Cada tarefa recebe um coroutine scope isolado.
3. Dispatcher distribui execução.
4. Jobs são executados em paralelo com limites definidos.

UI + Background simultâneo

1. UI Runtime usa Main Dispatcher.
2. Scheduler usa IO Dispatcher.
3. Ambos coexistem sem bloqueio.

Cancelamento por bateria baixa

1. Power Manager detecta bateria crítica.
2. Threading Model ativa throttling.
3. Jobs de baixa prioridade são cancelados.
4. UI permanece ativa.

Sobrecarga do sistema

1. Muitos plugins ativos.
2. Dispatcher entra em modo throttled.

- 3. Scheduler reduz concorrência.
- 4. Jobs são reordenados.

12. Regras Arquiteturais

- Todo código concorrente deve usar o modelo oficial.
- Threads diretas não são permitidas para plugins.
- Cancelamento deve ser cooperativo sempre.
- Cada plugin possui isolamento de concorrência.
- UI sempre tem prioridade sobre background tasks.
- Scheduler não executa diretamente; apenas agenda.
- Execution Runtime é o único executor final.

13. Limitações

O Threading Model não é responsável por:

- lógica de negócio;
- armazenamento;
- UI rendering;
- gerenciamento de plugins;
- definição de estados globais;
- execução direta de plugins.

14. Futuras Extensões

O modelo poderá evoluir para:

- execução distribuída entre dispositivos Vertex;
- threads virtuais por plugin;
- predição de carga baseada em uso;
- escalonamento baseado em IA;
- isolamento por processo para plugins pesados;
- execução híbrida CPU/GPU para workloads específicos.

15. Resumo

O Threading / Concurrency Model do Vertex define uma camada unificada de execução concorrente baseada em corrotinas Kotlin, com controle centralizado, isolamento por plugin e integração profunda com Scheduler e Execution Runtime. Ele garante que o sistema

permaneça responsivo, eficiente e seguro mesmo sob múltiplas cargas simultâneas, especialmente em dispositivos com recursos limitados como relógios Wear OS.